

Network Flow

- Maximum Flow: Problem Formulation
- Maximum Flow: Upper Bound
- Maximum Flow: Adding flow along paths
- Residual Network and Augmenting Path
- Ford-Fulkerson Algorithm – Max-Flow-Min-Cut Theorem
- Edmond-Karp Algorithm
- Maximum Flow: Variants and Applications

IMDAD ULLAH KHAN

Max Flow: Problem Formulation

Input: A flow network $G = (V, E, c)$, $c : E \rightarrow \mathbb{R}^+$

$s \in V$ a source and $t \in V$ a sink

$f : E \rightarrow \mathbb{R}^+$ ($f_e = f(e)$) is a flow if it satisfies

1 **capacity constraints** $\forall e \in E : 0 \leq f_e \leq c_e$

2 **flow conservation constraints** $\forall v \in V, v \neq s, t : f^{out}(v) = f^{in}(v)$

$$size(f) = f^{out}(s) = f^{in}(t)$$

Output: A flow f of maximum size

Let f be a flow in G and let $[A, \bar{A}]$ be any $s - t$ cut in G , then

$$size(f) \leq c([A, \bar{A}])$$

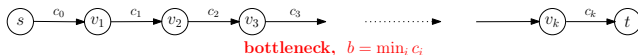
Max Flow: Adding Flow along a path

Greedy Algorithm – build up flow little bit at a time

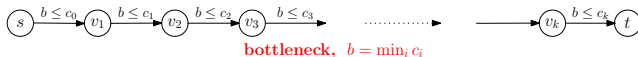
1 Start with a 0 flow

▷ Note: the flow f with $f_e = 0$ for all $e \in E$ satisfies both constraints

2 Add more flow to f via a $s - t$ path



Adding flow along a path keeps flow conservation constraints satisfied

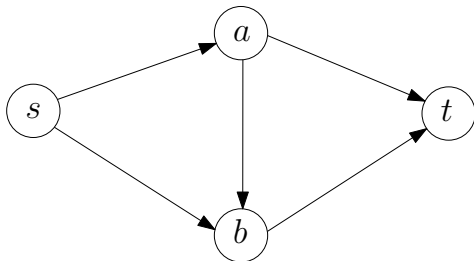


Adding flow equal to path-bottleneck keeps capacity constraints satisfied

The two ensure any intermediate flow by greedy algorithm is valid

Max Flow: Problems with the algorithm

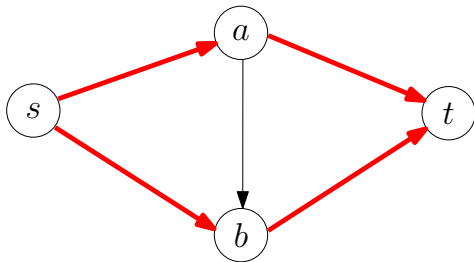
Consider the flow network – all edge capacities are 1



Max Flow: Problems with the algorithm

Consider the flow network – all edge capacities are 1

The max flow clearly is of size 2

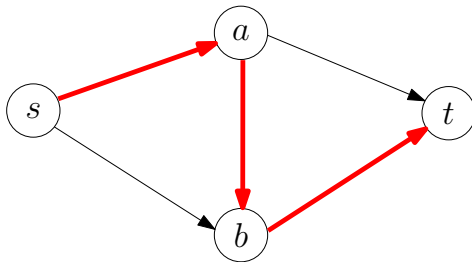


Max Flow: Problems with the algorithm

Consider the flow network – all edge capacities are 1

The max flow clearly is of size 2

If the greedy algorithm adds a flow of size 1 via the $s - t$ path s, a, b, t



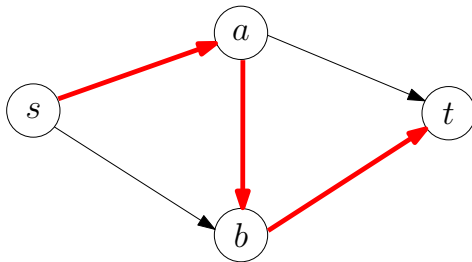
Max Flow: Problems with the algorithm

Consider the flow network – all edge capacities are 1

The max flow clearly is of size 2

If the greedy algorithm adds a flow of size 1 via the $s - t$ path s, a, b, t

No $s - t$ path in the remaining graph



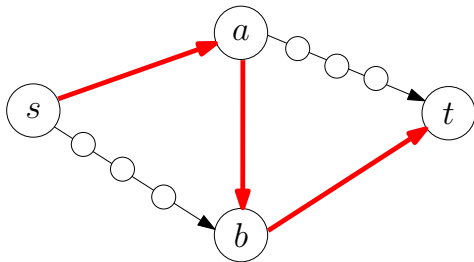
Max Flow: Problems with the algorithm

Consider the flow network – all edge capacities are 1

The max flow clearly is of size 2

If the greedy algorithm adds a flow of size 1 via the $s - t$ path s, a, b, t

No $s - t$ path in the remaining graph

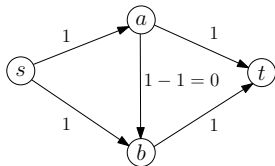
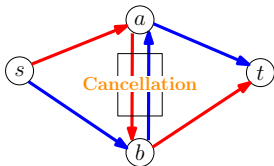
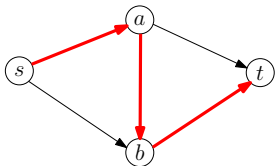


▷ The issue is not the hop-length of paths

Max Flow: Fix for the algorithm

A more general way of pushing further flow is to

- 1 Push forward flow on edges where some capacity is remaining
- 2 Cancel existing flow on edges ▷ pushing flow backward



- Add one unit of flow via the s, a, b, t path
- Add one unit of flow via the s, b, a, t path
 - ▷ $ba \notin E$, but we cancel the existing flow on edge $ab \in E$
- Add the red and blue flows

Cancellation of existing flows on edges (if need be) is the right framework to add more flow

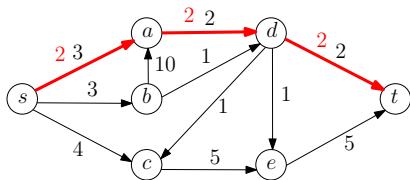
Residual network

- Provides, a systematic way to search for the right places to cancel flow and adding more flow
- Associated with a flow f , it encodes “places” where f can be increased (by adding new flows and canceling existing flows)

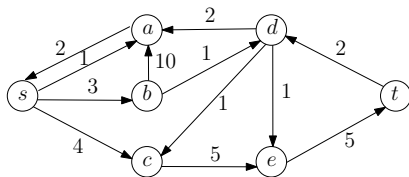
Max Flow: Residual Network

Given network G and flow f , the residual graph G_f of G w.r.t f is

- Vertex set of G_f is the same as that of G
- **Forward edges:** For each $e = uv$ of G on which $f_e < c_e$, there is an edge $e = uv$ in G_f with capacity $c_e - f_e > 0$
- **Backward edges:** For each edge $e = uv$ of G on which $f_e > 0$, there is an edge $e' = vu$ in G_f with capacity f_e



Flow network with flow shown in red



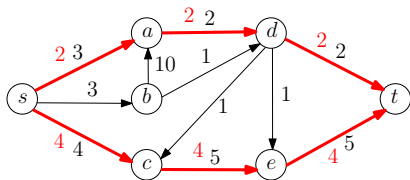
The corresponding residual network

▷ For any G and f , G_f has at most twice as many edges as G

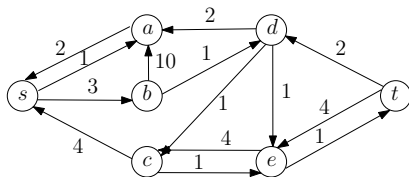
Max Flow: Residual Network

Given network G and flow f , the residual graph G_f of G w.r.t f is

- Vertex set of G_f is the same as that of G
- **Forward edges:** For each $e = uv$ of G on which $f_e < c_e$, there is an edge $e = uv$ in G_f with capacity $c_e - f_e > 0$
- **Backward edges:** For each edge $e = uv$ of G on which $f_e > 0$, there is an edge $e' = vu$ in G_f with capacity f_e



Flow network with flow shown in red



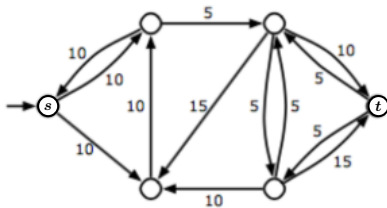
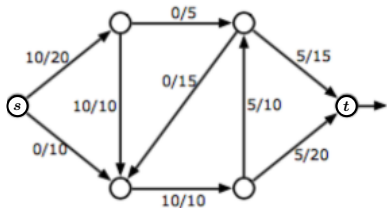
The corresponding residual network

▷ For any G and f , G_f has at most twice as many edges as G

Max Flow: Residual Network

Given network G and flow f , the residual graph G_f of G w.r.t f is

- Vertex set of G_f is the same as that of G
- **Forward edges:** For each $e = uv$ of G on which $f_e < c_e$, there is an edge $e = uv$ in G_f with capacity $c_e - f_e > 0$
- **Backward edges:** For each edge $e = uv$ of G on which $f_e > 0$, there is an edge $e' = vu$ in G_f with capacity f_e



▷ For any G and f , G_f has at most twice as many edges as G

Max Flow: Augmenting Path

An **augmenting path** is a simple $s - t$ path in the residual graph G_f

▷ It is used to augment the flow f

For an augmenting path P in G_f , $bottleneck(P, f) = \min_{e \in P} c'_e$

▷ the minimum residual capacity of any edge on P in G_f

Note c'_e is the residual capacity of the edge e (its capacity in G_f)

Algorithm $AUGMENT(P, f)$ augment flow using a path P in G_f

$b \leftarrow bottleneck(P, f)$

$f' \leftarrow f$

for each edge $e = uv \in P$ **do**

if e is a forward edge **then**

$f'_e \leftarrow f_e + b$

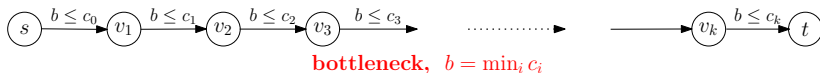
else if e is a backward edge **then**

$f'_{vu} \leftarrow f_{vu} - b$

Max Flow: Correctness of AUGMENT(P, f)

The output f' of AUGMENT(P, f) is a flow

f' satisfies capacity constraints $\forall e \in E : 0 \leq f'_e \leq c'_e$



- **Case 1:** e is not on P , $f'_e = f_e$
- **Case 2:** If $e \in P$ is a forward edge ($c'_e = c_e - f_e$), then $f'_e = f_e + b$
 - Since $0 < b \leq c'_e = c_e - f_e$
 - We have $0 \leq f_e \leq f'_e = f_e + b \leq f_e + c_e - f_e = c_e$
- **Case 3:** If $e \in P$ is a backward edge ($c'_e = f_e$), then $f'_e = f_e - b$
 - Since $0 < b \leq c'_e = f_e$
 - We have $c_e \geq f_e \geq f'_e = f_e - b \geq f_e - f_e = 0$

Max Flow: Correctness of AUGMENT(P, f)

The output f' of AUGMENT(P, f) is a flow

f' satisfies the flow conservation $\forall v \in V, v \neq s, t \quad f'^{out}(v) = f'^{in}(v)$



■ **Case 1:** e_{in} is forward edge and e_{out} is forward edge

■ $f'^{out}(v) = f^{out}(v) + b$ as $f'_{vy} = f_{vy} + b$ and

$f'^{in}(v) = f^{in}(v) + b$ as $f'_{xv} = f_{xv} + b$

■ Hence change in in-flow to v = change in out-flow from v

■ **Case 2:** e_{in} is reverse edge and e_{out} is forward edge

■ $f'^{out}(v) = f^{out}(v) - b + b$ as $f'_{vx} = f_{vx} - b$ (since vx is a reverse edge)
and $f'_{vy} = f_{vy} + b$ (since vy is a forward edge)

■ Hence $f'^{out}(v) = f'^{in}(v)$

■ **Case 3:** e_{in} is reverse edge and e_{out} is reverse edge

■ **Case 4:** e_{in} is forward edge and e_{out} is reverse edge

Network Flow

- Maximum Flow: Problem Formulation
- Maximum Flow: Upper Bound
- Maximum Flow: Adding flow along paths
- Residual Network and Augmenting Path
- Ford-Fulkerson Algorithm – Max-Flow-Min-Cut Theorem
- Edmond-Karp Algorithm
- Maximum Flow: Variants and Applications

IMDAD ULLAH KHAN

Max Flow: Augmenting Path

An **augmenting path** is a simple $s - t$ path in the residual graph G_f

▷ It is used to augment the flow f

For an augmenting path P in G_f , $bottleneck(P, f) = \min_{e \in P} c'_e$

▷ the minimum residual capacity of any edge on P in G_f

Note c'_e is the residual capacity of the edge e (its capacity in G_f)

Algorithm AUGMENT(P, f) augment flow using a path P in G_f

$b \leftarrow bottleneck(P, f)$

$f' \leftarrow f$

for each edge $e = uv \in P$ **do**

if e is a forward edge **then**

$f'_e \leftarrow f_e + b$

else if e is a backward edge **then**

$f'_{vu} \leftarrow f_{vu} - b$

Max Flow: The Ford-Fulkerson Algorithm

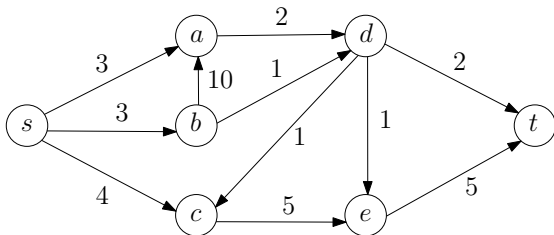
The Ford-Fulkerson algorithm repeatedly augments the current flow until the flow cannot be improved further

Algorithm Ford-Fulkerson Algorithm (G)

```
 $f \leftarrow 0$  ▷ Initialize to a (valid) flow of size 0 (on every edge)  
while TRUE do  
    Compute  $G_f$   
    Find an  $s - t$  path  $P$  in  $G_f$  ▷ Using e.g. DFS  
    if no such path then  
        return  $f$   
    else  
         $f \leftarrow \text{AUGMENT}(P, f)$ 
```

Max Flow: The Ford-Fulkerson Algorithm

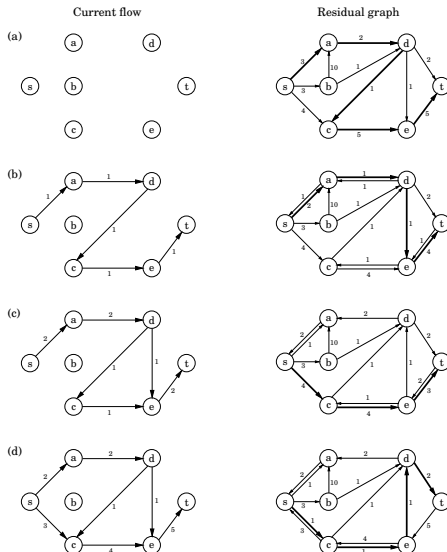
Executing the Ford-Fulkerson Algorithm on the following graph



diagrams taken from the DPV book

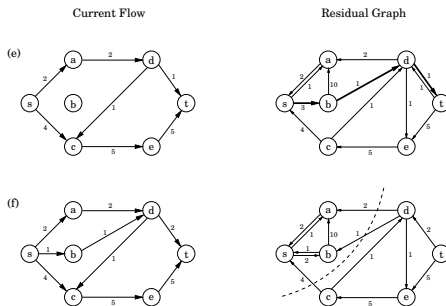
Max Flow: The Ford-Fulkerson Algorithm

Figure 7.6 The max-flow algorithm applied to the network of Figure 7.4. At each iteration, the current flow is shown on the left and the residual network on the right. The paths chosen are shown in bold.



Max Flow: The Ford-Fulkerson Algorithm

Figure 7.6 Continued



Max Flow: The Ford-Fulkerson Algorithm

The Ford-Fulkerson algorithm repeatedly augments the current flow

Algorithm Ford-Fulkerson Algorithm (G)

```
 $f \leftarrow 0$  ▷ Initialize to a (valid) flow of size 0 (on every edge)  
while TRUE do  
  Compute  $G_f$   
  Find an  $s - t$  path  $P$  in  $G_f$  ▷ Using e.g. DFS  
  if no such path then  
    return  $f$   
  else  
     $f \leftarrow \text{AUGMENT}(P, f)$ 
```

- “Correctness” follows from the correctness of the $\text{AUGMENT}(P, f)$
- We need to prove its termination
- We need to discuss its implementation and analyze its running time

Integrality:

If c_e is integer for every edge e in G , then for every intermediate flow f

- flow on every edge, f_e is integer
- capacity on every edge in G_f , c'_e is integer

Proof : *After iteration i , flow and capacity on all edges are integers*

Basis Step: After iteration 0, $\forall e \in E$, $f_e = 0$ and $c'_e \in \mathbb{Z}$ by construction

Inductive Hypothesis: Before iteration i , $\forall e \in E$, $f_e \in \mathbb{Z}$ and $c'_e \in \mathbb{Z}$

- The capacity b of the bottleneck edge on augmenting path P is integer
- $\forall e \in E \quad f_e^{(i)} = f_e^{(i-1)} \pm b \quad \triangleright \text{hence, remains } \in \mathbb{Z}$
- Also, $\forall e \in E$ in $G_{f^{(i)}}$, c'_e (of forward/backward edges) remains $\in \mathbb{Z}$

Flow is monotonically increasing:

Let f be a flow in G and let P be a $s - t$ path in G_f .

If f' is the flow returned by the $\text{AUGMENT}(P, f)$ function, then

$$\text{size}(f') = \text{size}(f) + b, \quad \text{where } b = \text{bottleneck}(P, f)$$

$$\text{size}(f) = f^{\text{out}}(s)$$

- P is $s - t$ path in $G_f \implies$ the first edge e on P is outgoing from s
- This edge sx must be a forward edge in G_f , (because if sx is a backward edge, then $xs \in E(G)$, contradicting $\deg^-(s) = 0$ in G)
- The AUGMENT procedure will make $f'(e) = f(e) + b$ hence
$$\text{size}(f') = f'^{\text{out}}(s) = f^{\text{out}}(s) + b = \text{size}(f) + b$$
- Since $b > 0$, we get that $\text{size}(f') > \text{size}(f)$

The Ford-Fulkerson Algorithm - Analysis

Termination: We only need to show that max flow is finite (bounded)

The algorithm terminates in at most $C_s = c([\{s\}, \overline{\{s\}}])$ steps

Let f be a flow in G and let $[A, \overline{A}]$ be any $s - t$ cut in G , then

$$\text{size}(f) \leq c([A, \overline{A}])$$

$$\text{size}(f) \leq C_s = c([\{s\}, \overline{\{s\}}])$$

In each iteration the flow increases by least an integer $b \geq 1$,

hence there can be at most C_s iterations

The Ford-Fulkerson Algorithm - Analysis

Implementation: $G = (V, E, c)$, $c : E \rightarrow \mathbb{Z}^+$, $|V| = n$, $|E| = m$

The Ford-Fulkerson algorithm can be implemented in $O(mC_s)$ time

Any G_f can have at most $2m$ edges

G_f can be constructed in $O(n + m)$

- We can find $s - t$ path in G_f in $O(n + m)$ ▷ BFS or DFS from s
- $\text{AUGMENT}(G, f)$ takes $O(n)$ ▷ incr/dec per edge of P
- At most C_s iterations ▷ flow increased by ≥ 1 in each iteration

This implementation takes $O(mC_s)$ time

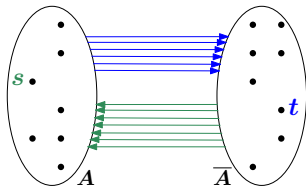
▷ assuming $m \geq n$

Max Flow: Upper Bound

Recall the following lemma we proved earlier

Let f be a flow in G and let $[A, \bar{A}]$ be any $s - t$ cut in G , then

$$\text{size}(f) \leq c([A, \bar{A}])$$



Tightest upper bound will come from a $s - t$ cut of minimum capacity

$[A^*, \bar{A}^*]$ be an $s - t$ cut with minimum capacity

▷ min- $s - t$ -cut

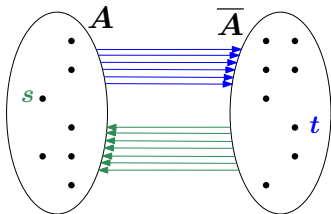
We get the corollary

$$\text{size}(f) \leq c([A^*, \bar{A}^*])$$

The Ford-Fulkerson Algorithm - Optimality

Let $[A, \bar{A}]$ be any $s - t$ cut. By definition we have

$$\begin{aligned} \text{size}(f) &= f^{\text{out}}(s) = f^{\text{out}}(s) + \sum_{s \neq v \in A} (f^{\text{out}}(v) - f^{\text{in}}(v)) && \text{just adding 0's} \\ &= f^{\text{out}}(s) + \sum_{s \neq v \in A} f^{\text{out}}(v) - \sum_{s \neq v \in A} f^{\text{in}}(v) \\ &= f^{\text{out}}(A) - f^{\text{in}}(A) \end{aligned}$$



$$\text{size}(f) = f^{\text{out}}(A) - f^{\text{in}}(A)$$

$$\text{size}(f) = f^{\text{in}}(\bar{A}) - f^{\text{out}}(\bar{A})$$

The Ford-Fulkerson Algorithm - Optimality

If f is a flow such that there is no $s - t$ path in G_f , then there is a $s - t$ cut $[A^*, \overline{A^*}]$ in G , such that $\text{size}(f) = c([A^*, \overline{A^*}])$

Construct such a cut. Let $A^* = \mathcal{R}(s)$ in G_f ▷ so $\overline{A^*} = \overline{\mathcal{R}(s)}$

$[A^*, \overline{A^*}]$ is $s - t$ cut, $s \in A^*$. No $s - t$ path in $G_f \implies t \in \overline{A^*}$

We show that for $e = xy$, $x \in A^*$, $y \in \overline{A^*}$, we have $f_e = c_e$

- If $f_e < c_e$, then $xy \in G_f$ with $c'_e = f_e - c_e > 0$. But then $y \in A^*$
 - ▷ All edges outgoing from A^* are saturated (no capacity left)

Similarly for $e = uv$, $u \in \overline{A^*}$ and $v \in A^*$, we have $f_e = 0$

- If $f_e > 0$, then $vu \in G_f$ with $c'_{vu} = f_e > 0$. But then $u \in A^*$
 - ▷ All edges incoming to A^* are completely unused

$$\text{size}(f) = f^{\text{out}}(A^*) - f^{\text{in}}(A^*) = \sum_{e \text{ outgoing from } A^*} c_e - \sum_{e \text{ incoming to } A^*} 0 = c([A^*, \overline{A^*}])$$

The Ford-Fulkerson Algorithm: Max-Flow-Min-Cut

Max-Flow-Min-Cut Theorem

If f is a flow with a corresponding cut $[A^*, \overline{A^*}]$ ($size(f) = c([A^*, \overline{A^*}])$), then f is a maximum flow and $[A^*, \overline{A^*}]$ is a minimum cut

Let f be a flow in G and let $[A, \overline{A}]$ be any $s - t$ cut in G , then

$$size(f) \leq c([A, \overline{A}])$$

- Immediate corollary to the above Lemma
- If there is flow of larger size than f
- Size of that flow is larger than the cut $c([A^*, \overline{A^*}])$
- A contradiction to the Lemma
- Similarly a cut of smaller capacity than $[A^*, \overline{A^*}]$ contradicts the Lemma

Theorem

The Ford-Fulkerson algorithm returns a maximum flow

Proof:

- It returns a flow f such that G_f has no $s - t$ path
- By the above theorem there is a cut with capacity equal to $\text{size}(f)$
- Hence by the Theorem f is optimal

Lemma

Given a maximum flow f in a network G , we can compute a minimum $s - t$ cut in G in $O(m)$ steps

Proof:

This minimum cut is given as a bonus

- 1 Run a DFS or BFS in G_f to find $\mathcal{R}(s)$
- 2 $[\mathcal{R}(s), \overline{\mathcal{R}(s)}]$ is a min-cut
- 3 G_f can be computed in $O(n + m) = O(m)$

A BFS or DFS in G_f takes at most $O(n + m) = O(m)$ time

Network Flow

- Maximum Flow: Problem Formulation
- Maximum Flow: Upper Bound
- Maximum Flow: Adding flow along paths
- Residual Network and Augmenting Path
- Ford-Fulkerson Algorithm – Max-Flow-Min-Cut Theorem
- Edmond-Karp Algorithm
- Maximum Flow: Variants and Applications

IMDAD ULLAH KHAN

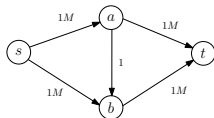
The Ford-Fulkerson Algorithm - Slow Example

The Ford-Fulkerson algorithm can be implemented in $O(mC_s)$ time, where $C_s = c([\{s\}, \overline{\{s\}}])$

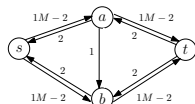
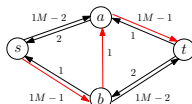
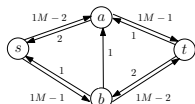
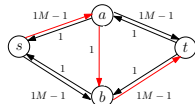
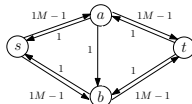
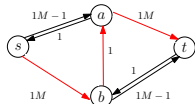
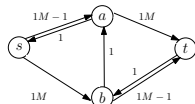
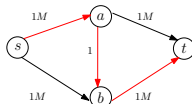
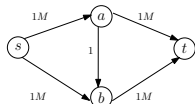
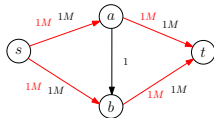
A tighter upper bound on runtime is $O(m \cdot C^*) = O(m \cdot \text{size}(f))$ where C^* is capacity of min cut

- The size of max flow f can be arbitrarily large
- not size of the input
- May be this upper bound is very loose upper (never met)

The Ford-Fulkerson Algorithm - Slow Example



Flow Network



Max Flow of
size $2M$

- Could take $2M$ iterations, augmenting 1 unit of flow
- No restriction on the choice of augmenting path
- Choose the augmenting path wisely to fix the problem

Max Flow: The Edmond-Karp Algorithm

Fix to Ford-Fulkerson algorithm: Choose shortest augmenting paths

Given a flow network G with source s and t

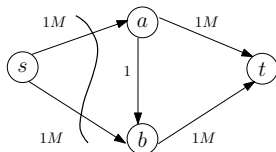
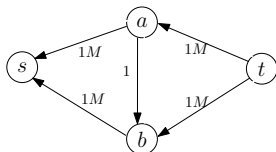
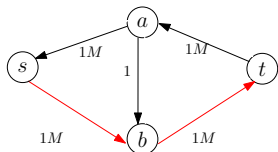
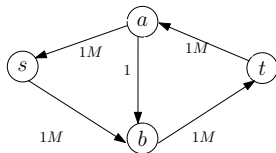
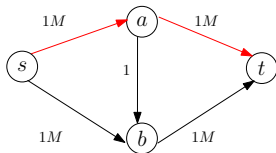
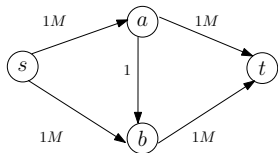
Algorithm Edmond-Karp Algorithm

```
 $f \leftarrow 0$  ▷ Initialize to a (valid) flow of size 0 (on every edge)  
while TRUE do  
  Compute  $G_f$   
  Find a shortest  $s - t$  path  $P$  in  $G_f$  ▷ Using BFS  
  if no such path then  
    return  $f$   
  else  
     $f \leftarrow \text{AUGMENT}(P, f)$ 
```

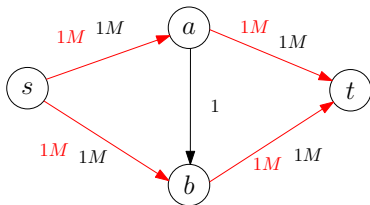
- Correctness and optimality follow immediately from the above proof, as that works for any augmenting path
- Need to analyze running time

Max Flow: The Edmond-Karp Algorithm

Running the Edmond-Karp algorithm on the pathological example



The Edmond-Karp Algorithm: Analysis



- Each $\text{AUGMENT}(P, f)$ saturates at least one edge on P
- The saturated edge is unavailable for the next iteration (possibly many iterations), at least in that direction

The key is to prove that an edge is not saturated too many times

▷ Main problem in the pathological example, the edge ab was saturated too many times

The Edmond-Karp Algorithm: Increasing Distances

After each iteration of the Edmonds-Karp algorithm, $d_{G_f}(s, v)$ (hop-length in G_f from s to v) does not decrease for all vertices v .

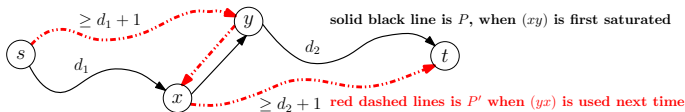
Same is true for $d_{G_f}(v, t)$ and hence $d_{G_f}(s, t)$.

Suppose the edge xy is saturated in G_f and f' is the augmented flow

- When xy is saturated we remove it from $G_{f'}$ ▷ edge yx is there
- $d_{G_f}(s, y)$ is greater than $d_{G_f}(s, x)$ ▷ sup-path optimality
 - if there was a shorter $s - y$ path we will use that to go to t
- In $G_{f'}$, the edge xy is not present, there cannot be a shorter path from s to v (a backward edge cannot decrease distance, as it is from a vertex farther away from s (e.g. y) to a vertex closer to s (e.g. x))
- All edges ever added are from vertices farther from s to vertices closer to s (we only introduce reverse edges along a shortest path)

The Edmond-Karp Algorithm: Edge Reuse

When edge $e = xy$ is saturated it is not reused until $d_{G_f}(s, t)$ strictly increases



- Suppose $e = xy$ is saturated when augmenting $P = s, \dots, x, y, \dots, t$
- P is a shortest $s - t$ path, $\ell(P) = d(s, t)$
- Let $d_1 = \ell(P[s - x])$ and $d_2 = \ell(P[y - t])$ $\triangleright \ell(P) = d_1 + 1 + d_2$
- Note that in this G_f $d(s, y) = d_1 + 1$
- Suppose yx is used in some $G_{f'}$ (since xy doesn't exist any more, for xy to be used again, yx must be used before it)
- By the previous lemma $d_{G_{f'}}(s, y) \geq d_1 + 1$ and $d_{G_{f'}}(x, t) \geq d_2 + 1$
- If a path in $G_{f'}$ uses yx , since it is a shortest path,
 $d_{G_{f'}}(s, t) \geq d_1 + 1 + 1 + d_2 + 1 \geq d_1 + d_2 + 3 \geq \ell(P) + 2$

Hence to use xy , distance from s to t must have increased



The Edmond-Karp Algorithm: Edge Reuse

The Edmond-Karp algorithm takes $O(nm^2)$ time

- Distance from s to t can only increase $O(n)$ times ▷ why?
- For a fixed value $d(s, t)$, an edge xy can be saturated only once
- After xy is saturated once, if the distance does not increase, then the algorithm can saturate other edges but not xy
- Hence for this fixed value of distance there can be at most $O(m)$ iterations
▷ each iteration saturates some edge
- After which the distance must increase, and for this second fixed value again there could be $O(m)$ iterations after which the distance must increase
- Number of different values of $d(s, t)$ is $O(n)$ so there could be $O(nm)$ iterations
- Each iteration takes $O(n + m)$ (BFS). Hence total runtime is $O(nm^2)$
▷ could be $O(n^5)$ for dense graphs. But is polynomial in size of input

See what happens if all capacities are 0's or all are 1's

Max Flow: Better Algorithms

- There are many better algorithms (please read about them)
- Some algorithms are directly based on the Ford-Fulkerson algorithm
 - ▷ like Edmond-Karp algorithm
- Especially, read the push-relabel algorithm in your textbook
- You are also encouraged to read about the scaling algorithm. This is a very useful trick
- The state of the art algorithm for Max-Flow algorithm is $O(nm)$